

Interview Question:

An Auto Loader stream keeps reprocessing files after a restart.

My Answer:

I'd inspect the checkpoint location first because Auto Loader uses checkpoint state to track progress. I'd verify that the checkpoint is stable, unique to the stream and source, and not being deleted by lifecycle rules. I'd also check whether the deployment accidentally changed the checkpoint path. After correcting the configuration, I'd test a controlled restart and compare processed-file counts so I know the fix works. I compare the before-and-after metrics, confirm the business result is unchanged, and monitor the next few production runs rather than treating one successful run as proof.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd compare run duration, input/output counts, micro-batch or task metrics, failures, retries, and backlog where applicable. I'd want to see the improvement across multiple runs.

Interview Question:

A production Auto Loader pipeline has increasing file-discovery latency.

My Answer:

I'd check how files are being discovered and whether file events are enabled for the workload. Current Databricks guidance recommends file-arrival triggering with file events for many production ingestion patterns because it can reduce unnecessary directory scanning. I'd also inspect source file counts and checkpoint health. I'd measure discovery latency separately from transformation latency so the optimization targets the real bottleneck. I use the execution evidence to identify the bottleneck, make one controlled change, and then compare runtime, resource usage, and output reconciliation against the original run.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd use the job run details, Spark UI, Delta history, and relevant streaming metrics to prove that the change is stable and that it did not introduce duplicates or missed data.

Interview Question:

A Delta MERGE is taking hours on a large table.

My Answer:

I'd first inspect the merge condition and the amount of data that can possibly match. If the source only contains a known date or business partition, I'd include that constraint in the match condition so Delta does not search the entire target unnecessarily. I'd also check file layout and concurrent writes. Then I'd compare files scanned, files rewritten, and total runtime after the change. For Databricks, After the change, I validate it with a representative production-sized workload. I check both technical metrics and data correctness, because a faster pipeline is not an improvement if it changes the expected business result.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd validate the target table state, processing latency, and operational metrics over several production runs. A stable result is more important than one successful execution.

Interview Question:

A streaming job fails because the checkpoint directory is missing.

My Answer:

I'd treat the checkpoint as critical state, not as temporary output. I'd confirm whether the path was deleted, changed between deployments, or attached to ephemeral storage. If the checkpoint is genuinely lost, restarting can require reprocessing depending on the source and semantics, so I'd first determine the safe replay boundary. For future deployments, I'd put checkpoint state in durable storage with appropriate access controls. For Databricks, I would document the root cause, the change, and the validation result in the runbook. That makes the fix repeatable for the team and gives the next engineer a clear way to troubleshoot the same pattern.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd compare run duration, input/output counts, micro-batch or task metrics, failures, retries, and backlog where applicable. I'd want to see the improvement across multiple runs.

Interview Question:

A Delta table has many small files after frequent micro-batches.

My Answer:

I'd inspect file size and write frequency first. Very frequent small commits can create small files, which increases metadata and read overhead. I'd consider whether the trigger frequency is unnecessarily aggressive and whether compaction or an appropriate optimization strategy is needed. I'd avoid manually manipulating Delta data files because the transaction log controls table consistency. I'd validate read performance after the change. For Databricks, I also keep rollback in mind. If the optimization or configuration change behaves differently at production scale, I can return to the previous safe configuration while investigating the new evidence.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd use the job run details, Spark UI, Delta history, and relevant streaming metrics to prove that the change is stable and that it did not introduce duplicates or missed data.

Interview Question:

A Delta table needs to support updates and inserts from a CDC feed.

My Answer:

I'd define the source operation types and the target business key first. Then I'd use a Delta merge pattern that maps inserts, updates, and deletes explicitly. I'd make the process idempotent so replaying the same CDC batch does not create duplicates. I'd also handle ordering if multiple changes for the same key can arrive in one batch. Finally, I'd reconcile source operation counts with target changes. For Databricks, My final check is not just elapsed time. I look at the relevant stage or query metrics, data volume, failures or retries, and reconciliation counts so the improvement is measurable and defensible in a production interview.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd validate the target table state, processing latency, and operational metrics over several production runs. A stable result is more important than one successful execution.

Interview Question:

A Databricks job works manually but fails when scheduled.

My Answer:

I'd compare the job context with the interactive notebook context: identity, cluster or compute type, parameters, workspace permissions, secrets, paths, and working directory assumptions. Scheduled jobs should not depend on notebook state left by a developer. I'd make parameters explicit and test the exact job configuration. I'd also capture logs and run IDs so the failure can be reproduced outside the UI session. I compare the before-and-after metrics, confirm the business result is unchanged, and monitor the next few production runs rather than treating one successful run as proof.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd compare run duration, input/output counts, micro-batch or task metrics, failures, retries, and backlog where applicable. I'd want to see the improvement across multiple runs.

Interview Question:

A team wants to store cloud credentials directly in a notebook.

My Answer:

I'd avoid hard-coding credentials because notebooks are shared artifacts and can expose secrets through source control or logs. I'd use the supported identity and secret-management pattern for the Azure Databricks environment, with Unity Catalog and managed identities where appropriate. I'd also apply least privilege and rotate credentials without changing application code. I'd test access using the same identity the production job will run under. I use the execution evidence to identify the bottleneck, make one controlled change, and then compare runtime, resource usage, and output reconciliation against the original run.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd use the job run details, Spark UI, Delta history, and relevant streaming metrics to prove that the change is stable and that it did not introduce duplicates or missed data.

Interview Question:

A Unity Catalog table is inaccessible to a job that worked previously.

My Answer:

I'd check the job identity and the table's catalog, schema, and object permissions first. I'd also verify that the compute and workspace are configured for the expected Unity Catalog environment. Rather than granting broad permissions, I'd identify the exact missing privilege and add only what the job needs. I'd then rerun with the production identity and document the access dependency. For Databricks, After the change, I validate it with a representative production-sized workload. I check both technical metrics and data correctness, because a faster pipeline is not an improvement if it changes the expected business result.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd validate the target table state, processing latency, and operational metrics over several production runs. A stable result is more important than one successful execution.

Interview Question:

A streaming query has growing state and memory usage.

My Answer:

I'd inspect whether the query is stateful and whether the state can be bounded. For event-time aggregations and joins, I'd evaluate whether a watermark is appropriate so old state can eventually be removed. I'd also review the input rate and micro-batch size. The goal is to bound state according to the business lateness requirement, then monitor state metrics after the change. For Databricks, I would document the root cause, the change, and the validation result in the runbook. That makes the fix repeatable for the team and gives the next engineer a clear way to troubleshoot the same pattern.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd compare run duration, input/output counts, micro-batch or task metrics, failures, retries, and backlog where applicable. I'd want to see the improvement across multiple runs.

Interview Question:

A stream is falling behind even though compute is available.

My Answer:

I'd compare input rate with processing rate and identify which stage is limiting throughput. More compute does not always help if the bottleneck is skew, a slow sink, file discovery, or a serial operation. I'd inspect micro-batch duration, input rows, processing time, and task distribution. Then I'd address the specific bottleneck and verify that backlog decreases over several batches. For Databricks, I also keep rollback in mind. If the optimization or configuration change behaves differently at production scale, I can return to the previous safe configuration while investigating the new evidence.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd use the job run details, Spark UI, Delta history, and relevant streaming metrics to prove that the change is stable and that it did not introduce duplicates or missed data.

Interview Question:

A Delta table query is slow because it scans too many files.

My Answer:

I'd inspect the table layout and the predicates used by the workload. I'd check whether file pruning or clustering can reduce the amount of data read, and whether the table has accumulated small or poorly organized files. I'd avoid assuming that adding a partition column is always the answer. I'd measure files scanned before and after the change and verify the query plan. For Databricks, My final check is not just elapsed time. I look at the relevant stage or query metrics, data volume, failures or retries, and reconciliation counts so the improvement is measurable and defensible in a production interview.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd validate the target table state, processing latency, and operational metrics over several production runs. A stable result is more important than one successful execution.

Interview Question:

A team is using ZORDER everywhere on new Databricks tables.

My Answer:

I'd first check the current table layout and workload because optimization should follow access patterns. Current Databricks guidance includes liquid clustering as a modern optimization approach and notes that older techniques should not be applied blindly. I'd identify the columns commonly used for selective filters and choose the supported layout strategy for the table. I'd validate query improvement against maintenance cost. I compare the before-and-after metrics, confirm the business result is unchanged, and monitor the next few production runs rather than treating one successful run as proof.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd compare run duration, input/output counts, micro-batch or task metrics, failures, retries, and backlog where applicable. I'd want to see the improvement across multiple runs.

Interview Question:

A Delta table is accidentally being modified outside Delta operations.

My Answer:

I'd stop the direct file manipulation and inspect the transaction log and table state. Delta expects data changes to be coordinated through the transaction log, so directly adding, deleting, or modifying Parquet files can corrupt table consistency. I'd restore the table using a supported recovery approach if necessary, then restrict storage access so applications cannot bypass the table interface. I use the execution evidence to identify the bottleneck, make one controlled change, and then compare runtime, resource usage, and output reconciliation against the original run.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd use the job run details, Spark UI, Delta history, and relevant streaming metrics to prove that the change is stable and that it did not introduce duplicates or missed data.

Interview Question:

A job's MERGE rewrites far more data than expected.

My Answer:

I'd inspect the merge condition and whether it is identifying the affected target range narrowly enough. I'd also check for duplicate source keys, which can complicate matching, and review the target file layout. Databricks recommends reducing the search space when known constraints are available. I'd measure target files scanned and rewritten, then confirm that the optimized condition still captures every valid update. For Databricks, After the change, I validate it with a representative production-sized workload. I check both technical metrics and data correctness, because a faster pipeline is not an improvement if it changes the expected business result.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd validate the target table state, processing latency, and operational metrics over several production runs. A stable result is more important than one successful execution.

Interview Question:

A streaming job produces duplicate output after a retry.

My Answer:

I'd first determine whether the source, transformation, or sink is responsible. For Delta-based streaming, checkpointing and supported streaming semantics provide strong guarantees, but custom side effects can still create duplicates. I'd avoid non-idempotent external writes without a deduplication strategy. I'd reproduce the retry and inspect batch IDs, checkpoints, and sink behavior before changing the transformation. For Databricks, I would document the root cause, the change, and the validation result in the runbook. That makes the fix repeatable for the team and gives the next engineer a clear way to troubleshoot the same pattern.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd compare run duration, input/output counts, micro-batch or task metrics, failures, retries, and backlog where applicable. I'd want to see the improvement across multiple runs.

Interview Question:

A Databricks pipeline must ingest JSON with evolving columns.

My Answer:

I'd use Auto Loader schema inference and evolution features rather than rebuilding the schema manually every time. I'd define a durable schema location and decide how unexpected fields should be handled, including whether rescued data is needed. I'd test an added column and an incompatible type change separately because those are different failure modes. I'd also monitor schema changes so they are visible to the team. For Databricks, I also keep rollback in mind. If the optimization or configuration change behaves differently at production scale, I can return to the previous safe configuration while investigating the new evidence.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd use the job run details, Spark UI, Delta history, and relevant streaming metrics to prove that the change is stable and that it did not introduce duplicates or missed data.

Interview Question:

A job needs to process all currently available files and then stop.

My Answer:

I'd use a triggered incremental pattern such as `Trigger.AvailableNow` when it matches the workload and runtime support. That allows the stream to process data available since the previous checkpoint and then exit. I'd keep the checkpoint stable between runs and schedule the job according to the required freshness. I'd validate that a second run processes only newly arrived data. For Databricks, My final check is not just elapsed time. I look at the relevant stage or query metrics, data volume, failures or retries, and reconciliation counts so the improvement is measurable and defensible in a production interview.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd validate the target table state, processing latency, and operational metrics over several production runs. A stable result is more important than one successful execution.

Interview Question:

A Databricks job has high cost but low data volume.

My Answer:

I'd separate compute time caused by actual processing from startup, idle, discovery, and over-provisioning. If the workload is incremental and minute-level latency is acceptable, an AvailableNow pattern can avoid continuously running compute. For file ingestion, file-arrival triggering may also reduce unnecessary runs. I'd compare cost per successful batch and latency before changing the architecture. I compare the before-and-after metrics, confirm the business result is unchanged, and monitor the next few production runs rather than treating one successful run as proof.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd compare run duration, input/output counts, micro-batch or task metrics, failures, retries, and backlog where applicable. I'd want to see the improvement across multiple runs.

Interview Question:

A pipeline needs data quality rules before publishing curated tables.

My Answer:

I'd place the checks at the boundary between raw and curated data so invalid records do not silently become trusted data. I'd define measurable rules for nulls, uniqueness, ranges, referential integrity, and schema. Records that fail can be quarantined when business rules allow it, while critical failures should stop publication. I'd persist quality metrics so operations can see whether the source is degrading. I use the execution evidence to identify the bottleneck, make one controlled change, and then compare runtime, resource usage, and output reconciliation against the original run.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd use the job run details, Spark UI, Delta history, and relevant streaming metrics to prove that the change is stable and that it did not introduce duplicates or missed data.

Interview Question:

A Databricks job has intermittent failures with no obvious code change.

My Answer:

I'd correlate failures with input batches, runtime version, compute events, external dependencies, and data characteristics. Intermittent failures often point to skew, transient infrastructure issues, rate limits, or non-deterministic assumptions. I'd use run IDs and logs to identify whether failures cluster around a stage or source. Once isolated, I'd add a targeted retry or workload fix rather than blanket retries. For Databricks, After the change, I validate it with a representative production-sized workload. I check both technical metrics and data correctness, because a faster pipeline is not an improvement if it changes the expected business result.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd validate the target table state, processing latency, and operational metrics over several production runs. A stable result is more important than one successful execution.

Interview Question:

A table needs historical versions for audit queries.

My Answer:

I'd use Delta's transaction history and time-travel capabilities where they meet the audit requirement, while being clear that retention and cleanup policies affect how far back historical data remains available. For regulatory retention, I would not rely solely on transient table history without confirming the required retention design. I'd test restoring or querying a historical version in a non-production environment. For Databricks, I would document the root cause, the change, and the validation result in the runbook. That makes the fix repeatable for the team and gives the next engineer a clear way to troubleshoot the same pattern.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd compare run duration, input/output counts, micro-batch or task metrics, failures, retries, and backlog where applicable. I'd want to see the improvement across multiple runs.

Interview Question:

A team wants to run VACUUM aggressively to reduce storage.

My Answer:

I'd first understand the table's recovery and time-travel requirements because cleanup removes old data files that are no longer needed by the table. I'd follow the platform's supported retention rules rather than choosing an arbitrary short interval. I'd also check whether long-running readers or downstream processes depend on older versions. Storage savings are useful only if they do not break the required recovery window. For Databricks, I also keep rollback in mind. If the optimization or configuration change behaves differently at production scale, I can return to the previous safe configuration while investigating the new evidence.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd use the job run details, Spark UI, Delta history, and relevant streaming metrics to prove that the change is stable and that it did not introduce duplicates or missed data.

Interview Question:

A Delta table has concurrent MERGE operations from multiple jobs.

My Answer:

I'd check whether the jobs can target overlapping data and whether the workload can be partitioned by a natural business boundary. Concurrent Delta operations provide transactional guarantees, but conflicting writes can still cause retries or failures. I'd reduce overlap where possible, make each operation idempotent, and monitor conflict rates. The goal is predictable concurrency rather than simply adding more parallel jobs. For Databricks, My final check is not just elapsed time. I look at the relevant stage or query metrics, data volume, failures or retries, and reconciliation counts so the improvement is measurable and defensible in a production interview.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd validate the target table state, processing latency, and operational metrics over several production runs. A stable result is more important than one successful execution.

Interview Question:

A Databricks notebook contains repeated SQL and Python transformations.

My Answer:

I'd refactor shared business logic into reusable functions, SQL views, or pipeline components where that improves maintainability. I'd keep data contracts and parameters explicit so reuse does not create hidden dependencies. For performance, I'd inspect whether repeated code is causing repeated scans or actions. Reuse should reduce both operational complexity and accidental differences between pipelines. I compare the before-and-after metrics, confirm the business result is unchanged, and monitor the next few production runs rather than treating one successful run as proof.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd compare run duration, input/output counts, micro-batch or task metrics, failures, retries, and backlog where applicable. I'd want to see the improvement across multiple runs.

Interview Question:

A job needs to promote code from development to production.

My Answer:

I'd package the pipeline so environment-specific values are parameters rather than hard-coded paths, catalog names, or credentials. I'd use version control and an automated deployment process, with validation in a lower environment before production. I'd also make the production identity and permissions explicit. After deployment, I'd run a small controlled validation and compare expected row counts and quality metrics. I use the execution evidence to identify the bottleneck, make one controlled change, and then compare runtime, resource usage, and output reconciliation against the original run.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd use the job run details, Spark UI, Delta history, and relevant streaming metrics to prove that the change is stable and that it did not introduce duplicates or missed data.

Interview Question:

A streaming query is stuck after a schema change.

My Answer:

I'd inspect the schema evolution behavior and the exact change that caused the failure. Adding a compatible column and changing an existing column's type are not equivalent. I'd review the checkpoint and schema-location state before deleting anything because that state affects recovery. I'd correct the schema contract or migration path, test it on representative data, and only then restart the production stream. For Databricks, After the change, I validate it with a representative production-sized workload. I check both technical metrics and data correctness, because a faster pipeline is not an improvement if it changes the expected business result.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd validate the target table state, processing latency, and operational metrics over several production runs. A stable result is more important than one successful execution.

Interview Question:

A Databricks pipeline needs to separate raw, refined, and serving data.

My Answer:

I'd keep the raw layer close to source fidelity, the refined layer focused on validated and standardized data, and the serving layer optimized for business consumption. With Unity Catalog, I'd use catalogs and schemas to organize ownership and permissions rather than relying only on folder names. I'd define data contracts and lineage between layers so users know what each table represents. For Databricks, I would document the root cause, the change, and the validation result in the runbook. That makes the fix repeatable for the team and gives the next engineer a clear way to troubleshoot the same pattern.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd compare run duration, input/output counts, micro-batch or task metrics, failures, retries, and backlog where applicable. I'd want to see the improvement across multiple runs.

Interview Question:

A query is slow because a UDF is used for simple string manipulation.

My Answer:

I'd replace the UDF with built-in Spark SQL functions when an equivalent exists. Built-in expressions can be optimized by the engine and avoid unnecessary Python execution overhead. I'd compare the physical plan and stage metrics before and after. I'd keep the UDF only if the business logic genuinely cannot be expressed with supported native functions. For Databricks, I also keep rollback in mind. If the optimization or configuration change behaves differently at production scale, I can return to the previous safe configuration while investigating the new evidence.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd use the job run details, Spark UI, Delta history, and relevant streaming metrics to prove that the change is stable and that it did not introduce duplicates or missed data.

Interview Question:

A large dimension table is joined to many fact tables.

My Answer:

I'd check whether the dimension is actually small enough for broadcasting in each workload; otherwise I'd focus on table layout, statistics, filtering, and join-key distribution. I'd project only required dimension columns and consider whether some attributes can be precomputed. I'd benchmark each consuming query because one join strategy may not be optimal for every fact table. For Databricks, My final check is not just elapsed time. I look at the relevant stage or query metrics, data volume, failures or retries, and reconciliation counts so the improvement is measurable and defensible in a production interview.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd validate the target table state, processing latency, and operational metrics over several production runs. A stable result is more important than one successful execution.

Interview Question:

A pipeline has a large backlog after a source outage.

My Answer:

I'd first estimate backlog size and required recovery time, then compare processing throughput with the incoming rate. I'd avoid changing multiple variables at once. I might temporarily increase parallelism or batch size if the workload supports it, while monitoring memory and sink capacity. After the backlog clears, I'd return to the normal configuration and document the recovery procedure. I compare the before-and-after metrics, confirm the business result is unchanged, and monitor the next few production runs rather than treating one successful run as proof.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd compare run duration, input/output counts, micro-batch or task metrics, failures, retries, and backlog where applicable. I'd want to see the improvement across multiple runs.

Interview Question:

A file source contains both valid and invalid records.

My Answer:

I'd keep ingestion resilient while preserving visibility into bad data. The parser should capture malformed records or rescued fields according to the source format, and I'd write a quality metric for the invalid percentage. If invalid data crosses a business threshold, the pipeline should alert or stop publication. I'd never silently discard the bad records because that makes reconciliation impossible. I use the execution evidence to identify the bottleneck, make one controlled change, and then compare runtime, resource usage, and output reconciliation against the original run.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd use the job run details, Spark UI, Delta history, and relevant streaming metrics to prove that the change is stable and that it did not introduce duplicates or missed data.

Interview Question:

A Unity Catalog external location works for one user but not a job.

My Answer:

I'd compare the effective identity and privileges rather than assuming the user's access represents the job's access. I'd verify access to the external location and the underlying storage configuration, then grant the minimum required permissions to the production principal. I'd test from the job context and document the dependency so future deployments do not fail unexpectedly. For Databricks, After the change, I validate it with a representative production-sized workload. I check both technical metrics and data correctness, because a faster pipeline is not an improvement if it changes the expected business result.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd validate the target table state, processing latency, and operational metrics over several production runs. A stable result is more important than one successful execution.

Interview Question:

A table needs efficient access by a frequently filtered business key.

My Answer:

I'd look at the actual query workload and data distribution before choosing a layout. The goal is to reduce unnecessary file reads while avoiding excessive maintenance or fragmentation. Depending on the table and current Databricks capabilities, I'd evaluate clustering or other supported data-layout optimizations. I'd compare files scanned, latency, and maintenance cost using representative queries. For Databricks, I would document the root cause, the change, and the validation result in the runbook. That makes the fix repeatable for the team and gives the next engineer a clear way to troubleshoot the same pattern.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd compare run duration, input/output counts, micro-batch or task metrics, failures, retries, and backlog where applicable. I'd want to see the improvement across multiple runs.

Interview Question:

A job writes successfully but downstream readers see stale results.

My Answer:

I'd determine whether the issue is caching, transaction visibility, orchestration order, or a downstream system reading a different table version. I'd inspect the write completion and job dependencies rather than adding arbitrary sleeps. If the downstream query uses caching, I'd validate cache behavior. I'd also use table history and run metadata to prove which version was committed and when. For Databricks, I also keep rollback in mind. If the optimization or configuration change behaves differently at production scale, I can return to the previous safe configuration while investigating the new evidence.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd use the job run details, Spark UI, Delta history, and relevant streaming metrics to prove that the change is stable and that it did not introduce duplicates or missed data.

Interview Question:

A team wants to use a single checkpoint for multiple streams.

My Answer:

I'd avoid that design. Each independent streaming query should have its own checkpoint state so progress and source metadata cannot be mixed. I'd give each stream a stable, unique checkpoint location and protect it from accidental deletion. If a stream is replaced, I'd decide explicitly whether it should continue from the existing state or start a new logical stream. For Databricks, My final check is not just elapsed time. I look at the relevant stage or query metrics, data volume, failures or retries, and reconciliation counts so the improvement is measurable and defensible in a production interview.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd validate the target table state, processing latency, and operational metrics over several production runs. A stable result is more important than one successful execution.

Interview Question:

A Delta table is queried by both batch and streaming workloads.

My Answer:

I'd define the consistency and freshness requirements for each reader. Delta supports both batch and streaming reads, but the operational design should account for retention, schema changes, and concurrent writes. I'd monitor long-running readers and ensure cleanup policies do not conflict with required streaming behavior. I'd also keep schema contracts stable so batch and streaming consumers do not break unexpectedly. I compare the before-and-after metrics, confirm the business result is unchanged, and monitor the next few production runs rather than treating one successful run as proof.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd compare run duration, input/output counts, micro-batch or task metrics, failures, retries, and backlog where applicable. I'd want to see the improvement across multiple runs.

Interview Question:

A production job must be observable without opening notebooks.

My Answer:

I'd make run status, input counts, output counts, duration, quality metrics, and failure reasons available through job monitoring and logs. For streaming, I'd track backlog and processing latency. I'd also include run IDs and batch identifiers in operational records. That lets support teams diagnose failures from telemetry rather than depending on a developer to inspect notebook output. I use the execution evidence to identify the bottleneck, make one controlled change, and then compare runtime, resource usage, and output reconciliation against the original run.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd use the job run details, Spark UI, Delta history, and relevant streaming metrics to prove that the change is stable and that it did not introduce duplicates or missed data.

Interview Question:

A pipeline needs to replay a historical date safely.

My Answer:

I'd make the date or batch range an explicit parameter and isolate the target scope. Before replaying, I'd determine whether the target should be replaced, merged, or versioned. The write must be idempotent so repeating the same replay does not duplicate records. I'd run the replay in a controlled environment first and reconcile the affected dates against source totals. For Databricks, After the change, I validate it with a representative production-sized workload. I check both technical metrics and data correctness, because a faster pipeline is not an improvement if it changes the expected business result.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd validate the target table state, processing latency, and operational metrics over several production runs. A stable result is more important than one successful execution.

Interview Question:

A Databricks workflow has dependencies between ingestion, transformation, and serv jobs.

My Answer:

I'd model those dependencies explicitly in the workflow rather than relying on manual execution order. Each task should have clear inputs, outputs, retry behavior, and failure handling. I'd make downstream tasks depend on successful completion and publish quality metrics before serving data. I'd also design the workflow so a failed downstream task can be rerun without unnecessarily reprocessing the entire ingestion layer. For Databricks, I would document the root cause, the change, and the validation result in the runbook. That makes the fix repeatable for the team and gives the next engineer a clear way to troubleshoot the same pattern.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd compare run duration, input/output counts, micro-batch or task metrics, failures, retries, and backlog where applicable. I'd want to see the improvement across multiple runs.

Interview Question:

A source schema change should not break downstream consumers unexpectedly.

My Answer:

I'd establish a schema contract and distinguish additive changes from breaking changes. For additive fields, I'd test whether downstream consumers safely ignore unknown columns. For breaking changes, I'd introduce a versioned contract or migration period. In Databricks, I'd also manage schema evolution deliberately rather than enabling broad automatic changes without monitoring. I'd test the exact source change before production. For Databricks, I also keep rollback in mind. If the optimization or configuration change behaves differently at production scale, I can return to the previous safe configuration while investigating the new evidence.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd use the job run details, Spark UI, Delta history, and relevant streaming metrics to prove that the change is stable and that it did not introduce duplicates or missed data.

Interview Question:

A Databricks pipeline needs least-privilege access.

My Answer:

I'd map every task to the data and compute resources it actually needs, then grant permissions at the catalog, schema, table, volume, or storage level as appropriate. I'd avoid sharing a powerful personal identity with production jobs. I'd test denied access as well as successful access so the boundary is real. With Unity Catalog, I'd keep governance and auditability central to the design. For Databricks, My final check is not just elapsed time. I look at the relevant stage or query metrics, data volume, failures or retries, and reconciliation counts so the improvement is measurable and defensible in a production interview.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd validate the target table state, processing latency, and operational metrics over several production runs. A stable result is more important than one successful execution.

Interview Question:

A streaming pipeline has an unexpected latency spike.

My Answer:

I'd break latency into input discovery, processing, state operations, and sink commit time. Then I'd compare the spike with input volume, file sizes, skew, schema changes, and downstream availability. If the issue is a temporary burst, I'd adjust processing capacity; if it is structural, I'd fix the slow stage. I'd keep a baseline of normal batch duration so future incidents are easier to diagnose. I compare the before-and-after metrics, confirm the business result is unchanged, and monitor the next few production runs rather than treating one successful run as proof.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd compare run duration, input/output counts, micro-batch or task metrics, failures, retries, and backlog where applicable. I'd want to see the improvement across multiple runs.

Interview Question:

A Delta table must support safe concurrent reads and writes.

My Answer:

I'd rely on Delta's transactional model rather than manipulating files directly. I'd also design writes so overlapping operations have predictable scopes and retry behavior. If conflicts occur frequently, I'd inspect which operations overlap and reduce unnecessary contention. The important validation is not only that individual writes succeed, but that readers consistently see valid table versions and no partial file-level state. I use the execution evidence to identify the bottleneck, make one controlled change, and then compare runtime, resource usage, and output reconciliation against the original run.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd use the job run details, Spark UI, Delta history, and relevant streaming metrics to prove that the change is stable and that it did not introduce duplicates or missed data.

Interview Question:

A team wants to optimize every table immediately.

My Answer:

I'd prioritize based on workload impact rather than optimizing tables that are rarely queried. I'd start with high-cost or high-frequency workloads, collect baseline metrics, apply one change, and measure the result. Current Databricks optimization features evolve, so I'd use the platform-supported approach for the runtime and table type instead of copying an old tuning recipe everywhere. For Databricks, After the change, I validate it with a representative production-sized workload. I check both technical metrics and data correctness, because a faster pipeline is not an improvement if it changes the expected business result.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd validate the target table state, processing latency, and operational metrics over several production runs. A stable result is more important than one successful execution.

Interview Question:

A Databricks job needs different configuration for dev and production.

My Answer:

I'd parameterize environment-specific values such as catalog, schema, paths, compute settings, and alert destinations rather than hard-coding them. I'd keep the code identical where possible and inject configuration at deployment or job-run time. Then I'd test the same workflow in both environments and verify that only the intended configuration changes. For Databricks, I would document the root cause, the change, and the validation result in the runbook. That makes the fix repeatable for the team and gives the next engineer a clear way to troubleshoot the same pattern.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd compare run duration, input/output counts, micro-batch or task metrics, failures, retries, and backlog where applicable. I'd want to see the improvement across multiple runs.

Interview Question:

A Delta table needs a controlled schema migration.

My Answer:

I'd treat the migration as a data contract change. I'd identify affected readers, test the new schema against representative data, and define whether the change is additive or breaking. I'd deploy the migration in a controlled sequence and monitor downstream failures. I'd avoid deleting or changing underlying files directly because Delta table consistency is managed through its transaction log. For Databricks, I also keep rollback in mind. If the optimization or configuration change behaves differently at production scale, I can return to the previous safe configuration while investigating the new evidence.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd use the job run details, Spark UI, Delta history, and relevant streaming metrics to prove that the change is stable and that it did not introduce duplicates or missed data.

Interview Question:

A Databricks pipeline must alert when data quality drops.

My Answer:

I'd define measurable thresholds such as invalid-record percentage, missing-key rate, or reconciliation variance. The pipeline would publish those metrics with each run and trigger an alert when a threshold is exceeded. I'd distinguish warning conditions from publication-blocking failures so the response matches business impact. I'd test the alert using deliberately bad test data. For Databricks, My final check is not just elapsed time. I look at the relevant stage or query metrics, data volume, failures or retries, and reconciliation counts so the improvement is measurable and defensible in a production interview.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd validate the target table state, processing latency, and operational metrics over several production runs. A stable result is more important than one successful execution.

Interview Question:

A streaming source suddenly sends much larger files.

My Answer:

I'd compare input file size, records per batch, processing time, and memory pressure with the normal baseline. Larger files can change micro-batch duration and task sizing even when the record count is stable. I'd tune ingestion limits or downstream partitioning only after identifying the bottleneck, then verify that backlog and memory remain stable. I compare the before-and-after metrics, confirm the business result is unchanged, and monitor the next few production runs rather than treating one successful run as proof.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd compare run duration, input/output counts, micro-batch or task metrics, failures, retries, and backlog where applicable. I'd want to see the improvement across multiple runs.

Interview Question:

A Databricks pipeline has a failed downstream task but successful ingestion.

My Answer:

I'd preserve the successfully ingested state and rerun only the failed downstream scope if the pipeline is designed to be idempotent. I'd inspect the failed task's input version and run ID before retrying. After recovery, I'd reconcile counts between ingestion and serving layers so the partial failure did not create duplicates or gaps. I use the execution evidence to identify the bottleneck, make one controlled change, and then compare runtime, resource usage, and output reconciliation against the original run.

Follow-up Question:

What Databricks operational metric or run evidence would you check to prove the fix is stable?

Follow-up Answer:

I'd use the job run details, Spark UI, Delta history, and relevant streaming metrics to prove that the change is stable and that it did not introduce duplicates or missed data.

Interview Question:

Auto Loader reprocesses files after deployment.

My Answer:

I'd check whether the checkpoint changed, was deleted, or is shared. Auto Loader uses Structured Streaming state, so the checkpoint must be durable and stable for that logical stream.

Follow-up Question:

Why is checkpoint state critical?

Follow-up Answer:

It records progress and recovery information; changing it can alter replay behavior.

Interview Question:

A Databricks job has skewed join tasks.

My Answer:

I'd inspect Spark UI and the physical plan. AQE can dynamically handle supported skewed shuffle joins, so I'd verify AQE behavior before adding manual hints.

Follow-up Question:

What does AQE do?

Follow-up Answer:

It can adapt the runtime plan using actual stage statistics, including supported skew handling.

Interview Question:

A Delta MERGE scans too much data.

My Answer:

I'd review the match condition and restrict the target search when the source gives a reliable date or key range. I'd also check duplicate source keys and target layout.

Follow-up Question:

How do you validate?

Follow-up Answer:

Compare files scanned, files rewritten, runtime, and unchanged business results.

Interview Question:

A streaming query has growing state.

My Answer:

I'd identify the stateful operation and define acceptable lateness. Where supported, event-time watermarks can bound state. I'd monitor state size and late-data behavior.

Follow-up Question:

What is the trade-off?

Follow-up Answer:

Tighter lateness limits control state but can exclude very late records from future results.

Interview Question:

A Databricks job needs secure ADLS access.

My Answer:

I'd use Unity Catalog-governed storage and the production identity rather than hard-coded credentials. I'd grant only required permissions and test with the job identity.

Follow-up Question:

Why avoid notebook credentials?

Follow-up Answer:

They can be exposed through source or logs and are harder to govern.

Interview Question:

A Unity Catalog table works for a user but not a workflow.

My Answer:

I'd compare effective identity and permissions, verify catalog/schema/table or storage access, and grant only the missing privilege.

Follow-up Question:

What principle applies?

Follow-up Answer:

Least privilege.

Interview Question:

A Databricks stream falls behind.

My Answer:

I'd compare input and processing rates and identify the slow stage. I'd inspect skew, discovery, state, and sink latency before increasing compute.

Follow-up Question:

Why might more compute not help?

Follow-up Answer:

The bottleneck can be a single skewed task, sink, or serial operation.

Interview Question:

A Delta table has many small files.

My Answer:

I'd inspect write frequency and file-size distribution and use supported optimization rather than direct file manipulation. I'd measure file count and query scan behavior.

Follow-up Question:

What should improve?

Follow-up Answer:

Fewer inefficient files and better scan behavior without breaking Delta consistency.

Interview Question:

A Databricks workflow needs a historical backfill.

My Answer:

I'd parameterize source range and target scope, test a small interval, and make the write idempotent. I'd reconcile before expanding the backfill.

Follow-up Question:

What makes replay safe?

Follow-up Answer:

Deterministic scope, idempotent write, and reconciliation.

Interview Question:

A production stream fails after schema change.

My Answer:

I'd classify the change as additive or breaking, inspect schema evolution and checkpoint state, and implement the approved migration before restarting.

Follow-up Question:

Why not reset checkpoint?

Follow-up Answer:

It can change recovery and replay behavior.

Interview Question:

A Databricks notebook repeats expensive actions.

My Answer:

I'd identify repeated actions and evaluate persistence only where it reduces recomputation enough to justify memory use. I'd remove accidental debug actions.

Follow-up Question:

How do you prove benefit?

Follow-up Answer:

Compare scans, stage duration, memory, and total runtime.

Interview Question:

A table query scans too many files.

My Answer:

I'd inspect filters, table layout, statistics, and supported clustering or optimization. I'd optimize based on real workload patterns rather than applying a generic recipe.

Follow-up Question:

What is the target?

Follow-up Answer:

Lower representative query latency and data scanned at reasonable maintenance cost.

Interview Question:

A pipeline needs schema evolution with governance.

My Answer:

I'd define allowed additive changes and require review for incompatible changes. I'd record schema changes and test downstream consumers.

Follow-up Question:

Why not allow everything?

Follow-up Answer:

A type or required-column change can break consumers even when ingestion succeeds.

Interview Question:

A production job needs a service identity.

My Answer:

I'd use a non-personal identity with minimum required permissions and test the exact production identity before release.

Follow-up Question:

Why avoid personal identities?

Follow-up Answer:

They make production dependent on an individual and complicate audit and rotation.

Interview Question:

A Databricks pipeline needs data-quality gates.

My Answer:

I'd make checks explicit before publishing trusted tables. I'd define rules for nulls, uniqueness, reconciliation, and schema and store results with the run ID.

Follow-up Question:

What is the benefit?

Follow-up Answer:

Successful publication becomes a meaningful data-quality signal.

Interview Question:

A streaming job has latency spikes.

My Answer:

I'd split latency into discovery, processing, state, and sink time, then compare the spike with input volume and task distribution.

Follow-up Question:

What baseline helps?

Follow-up Answer:

Normal batch duration, input rate, processing rate, and backlog.

Interview Question:

A Delta table needs audit history.

My Answer:

I'd use Delta history and time travel within the required retention period and separately define long-term audit retention if necessary.

Follow-up Question:

What limitation matters?

Follow-up Answer:

Historical availability depends on retention and cleanup settings.

Interview Question:

A workflow has ingestion, transformation, and serving tasks.

My Answer:

I'd model explicit dependencies and make quality checks prerequisites for serving. Each task needs clear inputs, outputs, retries, and recovery scope.

Follow-up Question:

Why explicit dependencies?

Follow-up Answer:

They make orchestration and partial recovery predictable.

Interview Question:

A pipeline receives malformed JSON.

My Answer:

I'd capture malformed or rescued data using the supported ingestion pattern and monitor the invalid rate rather than silently dropping records.

Follow-up Question:

What should be retained?

Follow-up Answer:

Source and batch metadata plus enough information to diagnose the malformed record.

Interview Question:

A job needs environment-specific configuration.

My Answer:

I'd parameterize catalog, schema, storage, compute, and alert settings instead of hard-coding them. The business logic should remain identical across environments.

Follow-up Question:

What is the benefit?

Follow-up Answer:

Safer promotion and fewer environment-specific code changes.

Interview Question:

Concurrent jobs write to one Delta table.

My Answer:

I'd identify overlapping write scopes and reduce unnecessary contention. Delta provides transactional behavior, but overlapping operations can still conflict.

Follow-up Question:

What should be monitored?

Follow-up Answer:

Write conflicts, retries, duration, and files rewritten.

Interview Question:

A pipeline needs efficient file discovery.

My Answer:

I'd evaluate file events and the supported production ingestion configuration rather than relying on unnecessary directory polling. I'd measure discovery latency and cost.

Follow-up Question:

What is the goal?

Follow-up Answer:

Efficient incremental detection without losing reliable checkpointed processing.

Interview Question:

A pipeline needs operational monitoring.

My Answer:

I'd capture duration, input/output counts, quality results, streaming backlog or latency, failures, and relevant table changes, all linked to run IDs.

Follow-up Question:

What makes monitoring useful?

Follow-up Answer:

It should show both technical and data/business health.

Interview Question:

Batch and streaming consumers read the same Delta table.

My Answer:

I'd define freshness, schema, retention, and concurrency requirements for both. I'd use supported Delta and Unity Catalog patterns and ensure cleanup policies do not break readers.

Follow-up Question:

What must be tested?

Follow-up Answer:

Reader compatibility, schema stability, retention, and consistent table behavior.

Interview Question:

A Databricks job has expensive Python UDFs.

My Answer:

I'd replace them with built-in Spark expressions or higher-order functions when possible and benchmark the result. I'd keep a UDF only when native functions cannot express the business rule.

Follow-up Question:

What is the benefit?

Follow-up Answer:

Less Python overhead and more opportunity for engine optimization.

Interview Question:

A Databricks table needs a better data layout.

My Answer:

I'd start with actual query predicates and table size, then use the current supported optimization approach rather than copying an old Z-ORDER recipe blindly.

Follow-up Question:

What should guide the choice?

Follow-up Answer:

The workload, runtime capabilities, and measurable scan reduction.

Interview Question:

A workflow retries a failed transformation.

My Answer:

I'd determine whether the failure is transient or deterministic before increasing retries. A deterministic schema or logic error should be fixed rather than repeatedly executed.

Follow-up Question:

When are retries useful?

Follow-up Answer:

For transient infrastructure or dependency failures where repetition can succeed.

Interview Question:

A pipeline must publish only after reconciliation.

My Answer:

I'd make reconciliation a workflow dependency and block the serving write when critical counts or business totals fail their threshold.

Follow-up Question:

Why block publication?

Follow-up Answer:

It prevents an operationally successful job from publishing untrusted data.

Interview Question:

A streaming job needs AvailableNow processing.

My Answer:

I'd use `Trigger.AvailableNow` when the workload needs to process currently available data and then stop, keeping the checkpoint stable between runs.

Follow-up Question:

What should the next run do?

Follow-up Answer:

It should continue from the checkpoint and process newly available data rather than restarting the full source.

Interview Question:

A Unity Catalog external table needs Auto Loader.

My Answer:

I'd keep the target table location and checkpoint in Unity Catalog-governed external locations and grant the required file and table permissions. I'd keep the checkpoint path separate from table data.

Follow-up Question:

What is important about the checkpoint location?

Follow-up Answer:

It should be durable, governed, and separate from the table data location.

Interview Question:

A Databricks query is slow after data growth.

My Answer:

I'd compare the physical plan, files scanned, shuffle, skew, and statistics with the earlier workload. I'd fix the measured bottleneck instead of simply scaling compute.

Follow-up Question:

What is the first comparison?

Follow-up Answer:

Input volume, files scanned, shuffle/stage duration, and execution plan.

Interview Question:

A Delta table needs concurrent reads and writes.

My Answer:

I'd rely on Delta transactional semantics and avoid direct file manipulation. I'd design write scopes to minimize conflicts and validate that readers see complete table versions.

Follow-up Question:

What should never be done?

Follow-up Answer:

Directly changing underlying table files outside supported table operations.

Interview Question:

A Databricks job needs data masking.

My Answer:

I'd use the platform governance and access-control features appropriate to the workspace and data classification. I'd test both permitted and denied access paths with production identities.

Follow-up Question:

What is the validation?

Follow-up Answer:

An authorized consumer sees the intended data while an unauthorized principal is denied or receives the governed representation.

Interview Question:

A stream has duplicate output after a retry.

My Answer:

I'd inspect checkpoint state, batch identifiers, and the sink behavior. Delta-based streaming can provide strong processing guarantees, but custom external side effects still need idempotency.

Follow-up Question:

What is the key check?

Follow-up Answer:

Determine whether duplication occurred in the stream transformation or in a non-idempotent downstream sink.

Interview Question:

A pipeline needs source file lineage.

My Answer:

I'd retain source file or batch metadata at ingestion and propagate it into the curated layer where useful. This supports reconciliation and incident investigation.

Follow-up Question:

Why preserve it?

Follow-up Answer:

It provides a direct link from target records to their ingestion source.

Interview Question:

A job needs cost control.

My Answer:

I'd measure compute startup, idle time, processing rate, and workload freshness requirements. Triggered incremental patterns can be more appropriate than continuously running compute for batch-like arrivals.

Follow-up Question:

What proves the change?

Follow-up Answer:

Cost per processed batch and freshness should improve without sacrificing required reliability.

Interview Question:

A pipeline needs a safe rollback.

My Answer:

I'd isolate outputs by batch or version, record the last valid state, and test the rollback procedure before production. The rollback must be reproducible rather than manual guesswork.

Follow-up Question:

What is the key design rule?

Follow-up Answer:

Always preserve a known-good recovery point.

Interview Question:

A Databricks pipeline has a large backlog after an outage.

My Answer:

I'd estimate backlog and required recovery time, then compare processing capacity with source rate. I'd scale or tune only the bottleneck and monitor backlog reduction.

Follow-up Question:

What should happen after recovery?

Follow-up Answer:

Return to the normal configuration and document the recovery procedure.

Interview Question:

A job uses a single checkpoint for multiple streams.

My Answer:

I'd separate checkpoint locations because each independent logical stream needs its own progress and source state.

Follow-up Question:

Why?

Follow-up Answer:

Shared checkpoint state can mix recovery metadata and make independent streams unsafe.

Interview Question:

A table is updated by multiple pipelines with overlapping keys.

My Answer:

I'd identify overlap and coordinate write scopes or orchestration. If concurrency is necessary, I'd make operations idempotent and monitor conflicts.

Follow-up Question:

What is the goal?

Follow-up Answer:

Predictable transactional behavior with minimal unnecessary contention.

Interview Question:

A Databricks workflow needs deployment promotion.

My Answer:

I'd parameterize environment values, version the code, test in a lower environment, and deploy through a repeatable process. I'd validate the production run with row counts and quality checks.

Follow-up Question:

What should never be hard-coded?

Follow-up Answer:

Production credentials, storage paths, and environment-specific catalog or schema values.

Interview Question:

A pipeline needs a clear data contract.

My Answer:

I'd document schema, grain, keys, freshness, quality rules, and allowed evolution. I'd enforce the contract at ingestion or publication boundaries.

Follow-up Question:

Why is grain important?

Follow-up Answer:

It tells consumers exactly what one row represents and prevents incorrect joins or aggregations.

Interview Question:

A Databricks deployment promotes the same notebook from test to production but must use different storage and compute settings.

My Answer:

I'd separate environment-specific configuration from transformation logic. Storage locations, catalog/schema names, checkpoint paths, and compute settings should be injected through deployment configuration rather than hard-coded in the notebook. I'd validate the rendered configuration before production.

Follow-up Question:

What would you test during promotion?

Follow-up Answer:

I'd verify target catalog and storage paths, identity and permissions, checkpoint locations, compute settings, and a controlled run against representative data.

Interview Question:

A Delta table has duplicate business keys after a retryable upstream failure.

My Answer:

I'd determine whether duplicates came from the source, retry behavior, or a non-idempotent write. I'd define the business key and intended record version, then use deterministic deduplication or MERGE logic. I'd reconcile the target before and after cleanup rather than deleting rows blindly.

Follow-up Question:

How would you prevent the problem from returning?

Follow-up Answer:

I'd make the write idempotent, persist a stable source or batch identifier, and add a data-quality rule that detects duplicate business keys before the serving layer.

Interview Question:

A Databricks streaming job needs a controlled historical backfill without disturbing the production stream.

My Answer:

I'd separate the backfill scope from the ongoing stream so the production checkpoint remains stable. I'd use an explicit input boundary for the historical load, reconcile the backfill, and let the production stream continue from its own checkpoint.

Follow-up Question:

Why keep separate checkpoints?

Follow-up Answer:

A checkpoint contains query progress and state. Reusing it across different logical streams can cause incorrect recovery, replay, or skipping behavior.

Interview Question:

A Delta table is read by many dashboards while a batch pipeline performs frequent writes.

My Answer:

I'd inspect the write pattern, file sizes, table layout, and query scan behavior. Delta supports concurrent reads and writes, but poor file layout or excessive small files can still hurt performance. I'd optimize based on measured workload evidence.

Follow-up Question:

What would you monitor?

Follow-up Answer:

I'd monitor query duration, files scanned, input bytes, write duration, file counts, and failed or retried operations before and after any layout change.

Interview Question:

A Databricks pipeline must apply row-level masking for sensitive columns while preserving approved access.

My Answer:

I'd use Unity Catalog governance and least-privilege access rather than duplicating masking logic in every notebook. I'd define which identities can see raw values and which consumers receive masked values, then test effective permissions with the production identities.

Follow-up Question:

Why test with the production identity?

Follow-up Answer:

A notebook author may have broader privileges than a job identity. Testing with the runtime identity verifies that the deployed workload has exactly the permissions it needs.

Interview Question:

A streaming pipeline falls behind because the sink is slower than the incoming data rate.

My Answer:

I'd separate source arrival rate, transformation time, state-store work, and sink latency. I'd check whether the sink supports the required write pattern, reduce unnecessary transformations, and tune capacity based on measured bottlenecks rather than blindly increasing compute.

Follow-up Question:

What metric best proves recovery?

Follow-up Answer:

I'd compare input rate with processed rate and track backlog over multiple batches. Recovery means the backlog trends down while correctness and sink reconciliation remain stable.

Interview Question:

A Databricks job must recover safely after an executor or cluster failure.

My Answer:

I'd rely on durable job configuration, checkpointing for streaming workloads, and idempotent batch writes. I'd confirm that the job can restart without duplicating or losing records, then test a controlled failure in non-production.

Follow-up Question:

What should be validated after recovery?

Follow-up Answer:

I'd reconcile source-to-target counts, business-key uniqueness, checkpoint progress where applicable, and the final Delta table version or output state.

Interview Question:

A Delta table has a schema change that is compatible for some consumers but breaking for others.

My Answer:

I'd treat schema evolution as a contract decision, not just a technical setting. I'd identify affected consumers, classify the change as additive or breaking, and introduce a compatible version or migration path when necessary. I'd validate downstream jobs before promotion.

Follow-up Question:

What makes a schema change breaking?

Follow-up Answer:

Changing a data type, removing a required column, changing semantics, or altering the grain can break consumers even when the table accepts the new schema.